

SET - II

1. College Examination System

Aim

To design and implement a College Examination System using student, subject, and marks tables with normalization up to 3NF, constraints, and result report generation.

Algorithm

1. Create STUDENT, SUBJECT, and MARKS tables.
 2. Apply primary key and foreign key constraints.
 3. Insert sample records.
 4. Normalize the database to 3NF.
 5. Generate student result reports using SQL queries.
 6. Display total, average, and grade.
-

MySQL Queries

```
CREATE DATABASE CollegeExam;
```

```
USE CollegeExam;
```

```
CREATE TABLE Student(
```

```
    SID INT PRIMARY KEY,
```

```
    SNAME VARCHAR(50),
```

```
    DEPT VARCHAR(30)
```

```
);
```

```
CREATE TABLE Subject(
```

```
    SUBID INT PRIMARY KEY,
```

```
    SUBNAME VARCHAR(50)
```

```
);
```

```
CREATE TABLE Marks(  
    SID INT,  
    SUBID INT,  
    MARKS INT CHECK(MARKS>=0 AND MARKS<=100),  
    PRIMARY KEY(SID,SUBID),  
    FOREIGN KEY(SID) REFERENCES Student(SID),  
    FOREIGN KEY(SUBID) REFERENCES Subject(SUBID)  
);
```

```
INSERT INTO Student VALUES  
(1,'Arun','CSE'),  
(2,'Priya','IT');
```

```
INSERT INTO Subject VALUES  
(101,'DBMS'),  
(102,'OS');
```

```
INSERT INTO Marks VALUES  
(1,101,85),  
(1,102,90),  
(2,101,78),  
(2,102,88);
```

```
SELECT S.SID,S.SNAME,  
SUM(M.MARKS) AS TOTAL,  
AVG(M.MARKS) AS AVERAGE  
FROM Student S
```

JOIN Marks M ON S.SID=M.SID

GROUP BY S.SID,S.SNAME;

Result

Thus the College Examination System was designed and implemented successfully.

2. Employee and Project Queries

Aim

To perform SQL queries on EMPLOYEE and PROJECT tables.

Algorithm

1. Create EMPLOYEE and PROJECT tables.
 2. Insert records.
 3. Execute aggregate and grouping queries.
-

MySQL Queries

```
CREATE TABLE Employee(  
    EID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    DEPT VARCHAR(30),  
    SALARY INT  
);
```

```
CREATE TABLE Project(  
    EID INT,  
    PID INT,  
    HOURS INT  
);
```

```
INSERT INTO Employee VALUES  
(1,'Ravi','CSE',50000),  
(2,'Anu','IT',60000),  
(3,'Kiran','CSE',70000);
```

INSERT INTO Project VALUES

**(1,101,20),
(1,102,25),
(2,103,30),
(3,104,40);**

-- Employees working on multiple projects

**SELECT EID,COUNT(PID) AS PROJECT_COUNT
FROM Project
GROUP BY EID
HAVING COUNT(PID)>1;**

-- Highest salary per department

**SELECT DEPT,MAX(SALARY) AS HIGHEST_SALARY
FROM Employee
GROUP BY DEPT;**

-- Average hours worked

**SELECT AVG(HOURS) AS AVG_HOURS
FROM Project;**

Result

Thus the required employee and project queries were executed successfully.

3. Clinic Management System

Aim

To develop a Clinic Management System with patient records, doctor assignments, triggers, and reports.

Algorithm

1. Create Patient and Doctor tables.
 2. Create Appointment table.
 3. Create trigger for appointment logging.
 4. Generate reports.
-

MySQL Queries

```
CREATE TABLE Doctor(  
    DID INT PRIMARY KEY,  
    DNAME VARCHAR(50),  
    SPECIALIZATION VARCHAR(30)  
);
```

```
CREATE TABLE Patient(  
    PID INT PRIMARY KEY,  
    PNAME VARCHAR(50),  
    AGE INT  
);
```

```
CREATE TABLE Appointment(  
    AID INT PRIMARY KEY,  
    PID INT,  
    DID INT,  
    APP_DATE DATE,
```

```
FOREIGN KEY(PID) REFERENCES Patient(PID),  
FOREIGN KEY(DID) REFERENCES Doctor(DID)  
);
```

```
CREATE TABLE Appointment_Log(  
    LOGID INT AUTO_INCREMENT PRIMARY KEY,  
    MESSAGE VARCHAR(100)  
);
```

```
DELIMITER //  
CREATE TRIGGER appointment_trigger  
AFTER INSERT ON Appointment  
FOR EACH ROW  
BEGIN  
    INSERT INTO Appointment_Log(MESSAGE)  
VALUES('Appointment Added');  
END//  
DELIMITER ;
```

```
SELECT * FROM Appointment_Log;
```

Result

Thus the Clinic Management System was implemented successfully.

4. Customer and Orders Queries

Aim

To perform queries on CUSTOMER and ORDERS tables.

Algorithm

1. Create CUSTOMER and ORDERS tables.
 2. Insert records.
 3. Execute customer-based queries.
-

MySQL Queries

```
CREATE TABLE Customer(  
    CID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    CITY VARCHAR(30)  
);
```

```
CREATE TABLE Orders(  
    OID INT PRIMARY KEY,  
    CID INT,  
    AMOUNT INT,  
    FOREIGN KEY(CID) REFERENCES Customer(CID)  
);
```

```
INSERT INTO Customer VALUES  
(1,'Arun','Chennai'),  
(2,'Divya','Madurai');
```

```
INSERT INTO Orders VALUES
```


(101,1,5000),

(102,1,3000);

-- Top customers

SELECT CID,SUM(AMOUNT) AS TOTAL

FROM Orders

GROUP BY CID

ORDER BY TOTAL DESC;

-- Customers with no orders

SELECT NAME

FROM Customer

WHERE CID NOT IN

(SELECT CID FROM Orders);

-- City-wise revenue

SELECT C.CITY,SUM(O.AMOUNT) AS REVENUE

FROM Customer C

JOIN Orders O ON C.CID=O.CID

GROUP BY C.CITY;

Result

Thus the customer and order queries were executed successfully.

5. Digital Wallet System

Aim

To implement a Digital Wallet System using TCL commands and ACID properties.

Algorithm

1. Create Wallet table.
 2. Insert wallet balances.
 3. Perform transactions using COMMIT and ROLLBACK.
-

MySQL Queries

```
CREATE TABLE Wallet(  
    ID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    BALANCE INT  
);
```

```
INSERT INTO Wallet VALUES  
(1,'Ravi',10000),  
(2,'Anu',8000);
```

```
START TRANSACTION;
```

```
UPDATE Wallet  
SET BALANCE=BALANCE-2000  
WHERE ID=1;
```

```
UPDATE Wallet  
SET BALANCE=BALANCE+2000
```

WHERE ID=2;

COMMIT;

SELECT * FROM Wallet;

Result

Thus the Digital Wallet transaction was implemented successfully.

6. Sales Aggregation

Aim

To perform aggregation operations on SALES table.

MySQL Queries

```
CREATE TABLE Sales(
```

```
    SID INT,
```

```
    PRODUCT VARCHAR(30),
```

```
    AMOUNT INT,
```

```
    REGION VARCHAR(30)
```

```
);
```

```
INSERT INTO Sales VALUES
```

```
(1,'Laptop',50000,'South'),
```

```
(2,'Mobile',30000,'North'),
```

```
(3,'Laptop',70000,'South');
```

```
-- Region wise aggregation
```

```
SELECT REGION,SUM(AMOUNT) AS TOTAL
```

```
FROM Sales
```

```
GROUP BY REGION;
```

```
-- Best selling products
```

```
SELECT PRODUCT,SUM(AMOUNT) AS SALES
```

```
FROM Sales
```

```
GROUP BY PRODUCT
```

```
ORDER BY SALES DESC;
```

-- Average revenue

```
SELECT AVG(AMOUNT) AS AVG_REVENUE
```

```
FROM Sales;
```

Result

Thus the aggregation queries were executed successfully.

7. School Management System

Aim

To implement attendance tracking and grading logic using PL/SQL.

MySQL Queries

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    ATTENDANCE INT,  
    MARKS INT  
);
```

```
INSERT INTO Student VALUES  
(1,'Arun',90,85),  
(2,'Priya',80,70);
```

```
SELECT *,  
CASE  
    WHEN MARKS >= 90 THEN 'A'  
    WHEN MARKS >= 75 THEN 'B'  
    ELSE 'C'  
END AS GRADE  
FROM Student;
```

Result

Thus the School Management System was implemented successfully.

8. Product Subqueries

Aim

To perform subqueries on PRODUCT table.

MySQL Queries

```
CREATE TABLE Product(  
    PID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    CATEGORY VARCHAR(30),  
    PRICE INT  
);
```

```
INSERT INTO Product VALUES  
(1,'Laptop','Electronics',50000),  
(2,'Mouse','Electronics',500),  
(3,'Table','Furniture',4000);
```

-- Products above average price

```
SELECT *  
FROM Product  
WHERE PRICE >  
(SELECT AVG(PRICE) FROM Product);
```

-- Least expensive products per category

```
SELECT CATEGORY,MIN(PRICE)  
FROM Product  
GROUP BY CATEGORY;
```

Result

Thus the subqueries were executed successfully.

9. E-Commerce Order System

Aim

To implement order workflows using stored procedures and triggers.

MySQL Queries

```
CREATE TABLE Orders(  
    OID INT PRIMARY KEY,  
    PRODUCT VARCHAR(30),  
    STATUS VARCHAR(20)  
);  
  
DELIMITER //  
CREATE PROCEDURE AddOrder(  
    IN id INT,  
    IN pname VARCHAR(30)  
)  
BEGIN  
    INSERT INTO Orders  
    VALUES(id,pname,'Pending');  
END//  
DELIMITER ;  
  
CALL AddOrder(1,'Laptop');  
  
SELECT * FROM Orders;
```

Result

Thus the E-Commerce Order System was implemented successfully.

10. Student-Class Mapping

Aim

To retrieve student-class mappings and identify unassigned students.

MySQL Queries

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(30)  
);
```

```
CREATE TABLE Class(  
    CID INT PRIMARY KEY,  
    CNAME VARCHAR(30)  
);
```

```
CREATE TABLE Enroll(  
    SID INT,  
    CID INT  
);
```

```
-- Student-Class Mapping  
SELECT S.NAME,C.CNAME  
FROM Student S  
JOIN Enroll E ON S.SID=E.SID  
JOIN Class C ON E.CID=C.CID;
```

```
-- Unassigned students
```

```
SELECT NAME
FROM Student
WHERE SID NOT IN
(SELECT SID FROM Enroll);
```

Result

Thus the join queries were executed successfully.

11. Staff Leave Management System

Aim

To implement leave validation using triggers and generate reports.

MySQL Queries

```
CREATE TABLE LeaveRequest(
```

```
    ID INT PRIMARY KEY,
```

```
    NAME VARCHAR(30),
```

```
    DAYS INT
```

```
);
```

```
DELIMITER //
```

```
CREATE TRIGGER leave_validation
```

```
BEFORE INSERT ON LeaveRequest
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.DAYS > 10 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Leave exceeds limit';
```

```
END IF;
```

```
END//
```

```
DELIMITER ;
```

```
INSERT INTO LeaveRequest VALUES
```

```
(1,'Arun',5);
```

```
SELECT * FROM LeaveRequest;
```

Result

Thus the Staff Leave Management System was implemented successfully.

12. Transaction Trigger System

Aim

To create triggers for auditing and preventing invalid transactions.

MySQL Queries

```
CREATE TABLE TransactionTable(
```

```
    TID INT PRIMARY KEY,
```

```
    AID INT,
```

```
    AMOUNT INT
```

```
);
```

```
CREATE TABLE Audit(
```

```
    ID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    MESSAGE VARCHAR(100)
```

```
);
```

```
DELIMITER //
```

```
CREATE TRIGGER transaction_audit
```

```
AFTER INSERT ON TransactionTable
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.AMOUNT <= 0 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Invalid Amount';
```

```
ELSE
```

```
INSERT INTO Audit(MESSAGE)
```

```
VALUES('Transaction Added');
```

```
END IF;
```

END//

DELIMITER ;

Result

Thus the transaction auditing system was implemented successfully.

13. Movie Ticket Booking System

Aim

To design and implement a Movie Ticket Booking System with constraints, triggers, and booking trend analysis.

Algorithm

- 1. Create MOVIE and BOOKING tables.**
 - 2. Apply primary key and foreign key constraints.**
 - 3. Insert booking details.**
 - 4. Create trigger to prevent overbooking.**
 - 5. Analyze booking trends using SQL queries.**
-

MySQL Queries

**CREATE DATABASE MovieBooking;
USE MovieBooking;**

**CREATE TABLE Movie(
 MID INT PRIMARY KEY,
 MNAME VARCHAR(50),
 TOTAL_SEATS INT
);**

**CREATE TABLE Booking(
 BID INT PRIMARY KEY,
 MID INT,
 CUSTOMER_NAME VARCHAR(50),
 SEATS_BOOKED INT,**

```
BOOKING_DATE DATE,  
FOREIGN KEY(MID) REFERENCES Movie(MID)  
);
```

```
INSERT INTO Movie VALUES  
(1,'Avengers',100),  
(2,'Leo',80);
```

```
DELIMITER //  
CREATE TRIGGER seat_validation  
BEFORE INSERT ON Booking  
FOR EACH ROW  
BEGIN  
DECLARE booked INT;
```

```
SELECT IFNULL(SUM(SEATS_BOOKED),0)  
INTO booked  
FROM Booking  
WHERE MID=NEW.MID;
```

```
IF booked + NEW.SEATS_BOOKED >  
  (SELECT TOTAL_SEATS FROM Movie WHERE MID=NEW.MID)  
THEN  
  SIGNAL SQLSTATE '45000'  
  SET MESSAGE_TEXT='Seats Not Available';  
END IF;  
END//  
DELIMITER ;
```

```
INSERT INTO Booking VALUES  
(101,1,'Arun',5,'2026-05-10');
```

```
-- Booking trend analysis  
SELECT MID,SUM(SEATS_BOOKED) AS TOTAL_BOOKINGS  
FROM Booking  
GROUP BY MID;
```

Result

Thus the Movie Ticket Booking System was implemented successfully.

14. Order Table Index Optimization

Aim

To improve query performance using indexing techniques on ORDERS table.

Algorithm

1. Create ORDERS table.
 2. Insert sample records.
 3. Execute queries before indexing.
 4. Create indexes on frequently searched columns.
 5. Compare performance after indexing.
-

MySQL Queries

```
CREATE TABLE Orders(  
  OID INT PRIMARY KEY,  
  CID INT,  
  PRODUCT_ID INT,  
  ORDER_DATE DATE,  
  AMOUNT INT,  
  STATUS VARCHAR(20)  
);
```

```
INSERT INTO Orders VALUES  
(1,101,1001,'2026-05-01',5000,'Delivered'),  
(2,102,1002,'2026-05-02',3000,'Pending'),  
(3,101,1003,'2026-05-03',7000,'Delivered');
```

```
-- Query before indexing  
EXPLAIN  
SELECT * FROM Orders  
WHERE CID=101;
```

```
-- Create Index  
CREATE INDEX idx_customer  
ON Orders(CID);
```

```
CREATE INDEX idx_orderdate
```

ON Orders(ORDER_DATE);

-- Query after indexing

EXPLAIN

SELECT * FROM Orders

WHERE CID=101;

Performance Improvement

- Query execution becomes faster.
 - Full table scan is reduced.
 - Index improves searching and filtering speed.
-

Result

Thus indexing and query optimization were performed successfully.

15. Customer Payment Analysis

Aim

To analyze payment records and optimize queries using indexing.

Algorithm

1. Create CUSTOMER and PAYMENT tables.
 2. Insert records.
 3. Write analytical queries.
 4. Create indexes for optimization.
 5. Analyze performance.
-

MySQL Queries

```
CREATE TABLE Customer(  
  CID INT PRIMARY KEY,  
  NAME VARCHAR(30),  
  CITY VARCHAR(30)  
);
```

```
CREATE TABLE Payment(  
  PID INT PRIMARY KEY,  
  CID INT,  
  AMOUNT INT,  
  PAYMENT_DATE DATE,  
  FOREIGN KEY(CID) REFERENCES Customer(CID)  
);
```

```
INSERT INTO Customer VALUES  
(1,'Arun','Chennai'),  
(2,'Priya','Madurai');
```

```
INSERT INTO Payment VALUES  
(101,1,5000,'2026-05-01'),  
(102,1,3000,'2026-05-02');
```

```
-- Customers with payments above threshold  
SELECT CID,SUM(AMOUNT) AS TOTAL_PAYMENT
```

```
FROM Payment
GROUP BY CID
HAVING SUM(AMOUNT) > 5000;
```

```
-- Customers with no payments
SELECT NAME
FROM Customer
WHERE CID NOT IN
(SELECT CID FROM Payment);
```

```
-- Create index
CREATE INDEX idx_payment
ON Payment(CID);
```

```
EXPLAIN
SELECT * FROM Payment
WHERE CID=1;
```

Performance Improvement

- Faster retrieval of payment details.
- Reduced search time using index.

Result

Thus payment analysis and optimization were completed successfully.

16. Employee Salary and Join Date Analysis

Aim

To analyze employee salary and recent joiners using indexing strategies.

Algorithm

1. Create EMPLOYEE table.
 2. Insert records.
 3. Execute salary and date queries.
 4. Create indexes on salary and join date.
 5. Compare query performance.
-

MySQL Queries

```
CREATE TABLE Employee(  
  EID INT PRIMARY KEY,  
  NAME VARCHAR(30),  
  DEPT VARCHAR(30),  
  SALARY INT,  
  JOIN_DATE DATE  
);
```

```
INSERT INTO Employee VALUES  
(1,'Arun','CSE',50000,'2025-01-10'),  
(2,'Priya','IT',60000,'2026-01-15'),  
(3,'Kavin','CSE',70000,'2026-03-01');
```

```
-- Department-wise average salary  
SELECT DEPT,AVG(SALARY) AS AVG_SALARY  
FROM Employee  
GROUP BY DEPT;
```

```
-- Recent joiners  
SELECT *  
FROM Employee  
WHERE JOIN_DATE > '2025-12-31';
```

```
-- Create indexes
```

```
CREATE INDEX idx_salary  
ON Employee(SALARY);
```

```
CREATE INDEX idx_joindate  
ON Employee(JOIN_DATE);
```

```
EXPLAIN  
SELECT * FROM Employee  
WHERE SALARY > 50000;
```

Indexing Strategy

- Salary index improves range-based salary search.
 - Date index improves recent employee filtering.
-

Result

Thus employee analysis and indexing were implemented successfully.

17. Student Attendance Management

Aim

To calculate attendance percentage, identify low attendance students, create views, and optimize queries using indexing.

Algorithm

1. Create STUDENT and ATTENDANCE tables.
 2. Insert attendance records.
 3. Calculate attendance percentage.
 4. Identify students below threshold.
 5. Create attendance summary view.
 6. Create indexes for optimization.
-

MySQL Queries

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    DEPT VARCHAR(30)  
);
```

```
CREATE TABLE Attendance(  
    SID INT,  
    ADATE DATE,  
    STATUS VARCHAR(10)  
);
```

```
INSERT INTO Student VALUES  
(1,'Arun','CSE'),  
(2,'Priya','IT');
```

```
INSERT INTO Attendance VALUES  
(1,'2026-05-01','Present'),  
(1,'2026-05-02','Absent'),  
(2,'2026-05-01','Present');
```

```
-- Attendance percentage
SELECT SID,
(SUM(CASE WHEN STATUS='Present' THEN 1 ELSE 0 END)
 / COUNT(*))*100 AS ATT_PERCENT
FROM Attendance
GROUP BY SID;

-- Students below threshold
SELECT SID
FROM Attendance
GROUP BY SID
HAVING (SUM(CASE WHEN STATUS='Present' THEN 1 ELSE 0 END)
 / COUNT(*))*100 < 75;

-- Create View
CREATE VIEW Attendance_Summary AS
SELECT SID,
COUNT(*) AS TOTAL_DAYS
FROM Attendance
GROUP BY SID;

-- Create Index
CREATE INDEX idx_attendance
ON Attendance(SID);
```

Result

Thus the attendance management system was implemented successfully.

18. Product and Stock Analysis

Aim

To identify top-selling categories, low stock products, and optimize queries using indexing.

Algorithm

1. Create PRODUCT table.
 2. Insert records.
 3. Perform analytical queries.
 4. Create indexes on frequently queried columns.
 5. Analyze performance improvement.
-

MySQL Queries

```
CREATE TABLE Product(  
  PID INT PRIMARY KEY,  
  NAME VARCHAR(30),  
  CATEGORY VARCHAR(30),  
  PRICE INT,  
  STOCK INT  
);
```

```
INSERT INTO Product VALUES  
(1,'Laptop','Electronics',50000,10),  
(2,'Mouse','Electronics',500,100),  
(3,'Chair','Furniture',3000,5);
```

```
-- Top selling categories  
SELECT CATEGORY,COUNT(*) AS TOTAL_PRODUCTS  
FROM Product  
GROUP BY CATEGORY  
ORDER BY TOTAL_PRODUCTS DESC;
```

```
-- Low stock products  
SELECT *  
FROM Product  
WHERE STOCK < 10;
```

```
-- Create indexes
CREATE INDEX idx_category
ON Product(CATEGORY);

CREATE INDEX idx_stock
ON Product(STOCK);

EXPLAIN
SELECT * FROM Product
WHERE CATEGORY='Electronics';
```

Performance Improvement

- Faster category-based searching.
 - Improved stock filtering performance.
-

Result

Thus product analysis and indexing were completed successfully.

19. Firebase-Based Chat Application

Aim

To develop a Firebase-based chat application with real-time updates.

Algorithm

1. Create Firebase project.
 2. Enable Firebase Realtime Database.
 3. Connect application with Firebase.
 4. Store and retrieve chat messages.
 5. Display real-time updates.
-

HTML Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Firebase Chat</title>
</head>
<body>

  <input type="text" id="msg">
  <button onclick="sendMessage()">Send</button>

  <ul id="messages"></ul>

  <script src="https://www.gstatic.com/firebasejs/8.6.8/firebase-app.js"></script>
  <script src="https://www.gstatic.com/firebasejs/8.6.8/firebase-database.js"></script>

  <script>

var firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "chatapp.firebaseio.com",
  databaseURL: "https://chatapp.firebaseio.com",
  projectId: "chatapp"
};
```

```
firebase.initializeApp(firebaseConfig);

var db = firebase.database();

function sendMessage(){
    var msg = document.getElementById("msg").value;

    db.ref("messages").push({
        text: msg
    });
}

db.ref("messages").on("child_added", function(data){

    var li = document.createElement("li");
    li.innerText = data.val().text;

    document.getElementById("messages").appendChild(li);
});

</script>

</body>
</html>
```

Result

Thus the Firebase chat application with real-time updates was developed successfully.

20. Sales Dataset Analytical Queries

Aim

To perform analytical queries on sales dataset to identify trends and seasonal patterns.

Algorithm

1. Create SALES table.
 2. Insert records.
 3. Perform monthly and yearly sales analysis.
 4. Identify seasonal sales patterns.
 5. Generate trend reports.
-

MySQL Queries

```
CREATE TABLE Sales(  
  SID INT PRIMARY KEY,  
  PRODUCT VARCHAR(30),  
  AMOUNT INT,  
  SALE_DATE DATE  
);
```

```
INSERT INTO Sales VALUES  
(1,'Laptop',50000,'2026-01-15'),  
(2,'Mobile',30000,'2026-02-20'),  
(3,'Laptop',70000,'2026-03-10'),  
(4,'TV',45000,'2026-03-25');
```

```
-- Monthly sales trend  
SELECT MONTH(SALE_DATE) AS MONTH,  
SUM(AMOUNT) AS TOTAL_SALES  
FROM Sales  
GROUP BY MONTH(SALE_DATE);
```

```
-- Product wise trend  
SELECT PRODUCT,  
SUM(AMOUNT) AS SALES  
FROM Sales  
GROUP BY PRODUCT
```

ORDER BY SALES DESC;

-- Seasonal pattern analysis

SELECT QUARTER(SALE_DATE) AS QUARTER,

SUM(AMOUNT) AS REVENUE

FROM Sales

GROUP BY QUARTER(SALE_DATE);

Result

Thus analytical queries were performed successfully to identify sales trends and seasonal patterns.